Web Crawling Strategy for High-Quality Training Data Collection

Thaluri Kavya
Student ID: U2423552H

1 Introduction

Modern large-scale AI systems, such as GPT, rely heavily on vast amounts of high-quality textual data for training. One primary method of acquiring such data is through web crawling, where automated agents systematically traverse the internet by following hyperlinks between pages. A crawler, such as GPTBot, typically begins from a set of seed URLs and incrementally discovers new pages through link exploration. However, the scale of the web presents a fundamental challenge: it is infeasible to crawl everything. Naive crawling strategies often result in the collection of redundant, low-quality, or noisy content, which can degrade the performance of downstream machine learning models.

Unlike traditional search engines, which aim for comprehensive coverage, AI training pipelines prioritise *data quality over quantity*. The objective is therefore not to crawl as many pages as possible, but rather to selectively acquire *useful, diverse, and reliable* textual data.

A key signal for estimating the importance of web pages is PageRank, a graph-based metric that captures the authority of a page based on the structure of incoming links. Pages with higher PageRank are generally more central, widely referenced, and informative. Leveraging such signals can significantly improve the quality of collected training data.

At the same time, ethical and legal considerations must be respected. Website owners can specify crawling permissions via `robots.txt`, which must be enforced by any responsible crawler.

This project proposes a ranking-based crawling strategy that replaces hand-tuned feature weights with **learned** weights estimated from data, producing a more principled and adaptable scoring function.

2 Problem Definition

Given a directed web graph and associated importance scores, the goal is to select the top-k most valuable pages for training data collection.

2.1 Motivation: Why High-PageRank Pages Are Valuable

High-PageRank pages are often strong candidates for training data due to the following properties:

- **Authority:** Pages referenced by many other pages are typically regarded as important sources of information.
- **Content quality:** Well-linked pages are more likely to contain comprehensive and coherent information.
- **Topic centrality:** Such pages often serve as hubs or summaries within a topic domain.
- **Reduced noise:** Isolated or weakly connected pages are more likely to contain spam or low-value content.

While PageRank is not a perfect indicator of quality, it provides a strong and scalable starting point that can be enhanced through additional learned signals.

Input:

- A directed graph $G = (V, E)$ representing URLs and hyperlinks
- Precomputed PageRank scores
- Crawl permissions derived from `robots.txt`
- Content quality priors (estimated per URL)

Objective: Select the top-k URLs that maximise training data quality.

This is fundamentally a **ranking problem**, rather than a traversal problem.

3 Methodology

3.1 Guiding Heuristic

We adopt the following guiding heuristic:

> Only crawl pages that are permitted, prioritise those whose features indicate high authority and content quality, and favour pages that facilitate further discovery.

Rather than hard-coding the balance between these signals, we **learn** the weights from data.

3.2 Feature Engineering

We construct an eight-dimensional feature vector for each crawl-allowed URL:

All continuous features are normalised to $[0, 1]$ using min-max scaling across the candidate set, ensuring no single feature dominates due to scale differences.

3.3 PageRank

PageRank captures global importance by propagating authority through the web graph:

$$PR(u) = \frac{1-d}{N} + d \sum_{v \in \mathrm{In}(u)} \frac{PR(v)}{|\mathrm{out}(v)|} \tag{1}$$

Feature	Symbol	Description
PageRank	$PR(u)$	Normalised global authority score
Content Quality	$CQ(u)$	Prior estimate of textual quality (0–1)
Inlink Score	$IL(u)$	Sum of $\frac{PR(v)}{\lvert\text{out}(v)\rvert}$ over all $v \to u$
In-degree	$ID(u)$	Normalised count of incoming edges
Domain Diversity	$DD(u)$	Normalised count of distinct linking domains
Out-degree	$OD(u)$	Normalised count of outgoing edges
Low-quality flag	$LQ(u)$	Binary: 1 if URL matches utility-page patterns
Low-trust flag	$LT(u)$	Binary: 1 if URL belongs to a low-trust domain

Table 1: Features used in the learned scoring function.

where $d = 0.85$ is the damping factor, N is the total number of pages, $\text{In}(u)$ is the set of pages linking to u, and $|\text{out}(v)|$ is the number of outgoing links from v.

3.4 Inlink Authority Score

Unlike raw in-degree, the inlink score $IL(u)$ weights each incoming link by the PageRank of the source divided by that source's out-degree. This provides a simplified HITS-style authority signal that captures both the number and the quality of incoming links:

$$IL(u) = \sum_{v\,:\,(v,u)\in E} \frac{PR(v)}{|\text{out}(v)|} \tag{2}$$

3.5 Weak-Supervision Label Generation

To train the weight model without hand-labelled data, we generate pseudo-labels using a transparent combination of quality signals:

$$\hat{y}(u) = 0.50\,CQ(u) + 0.30\,PR(u) + 0.10\,\mathbf{1}[\text{out-degree} > 0] - 0.30\,LQ(u) - 0.20\,LT(u) \tag{3}$$

These labels encode domain knowledge about what makes a page valuable for training data collection. The model then learns which observable features best predict $\hat{y}$, producing interpolated coefficients that generalise this signal. While the feature weights are learned from data, the supervision still depends on heuristic pseudo-label design.

3.6 Weight Learning via Ridge Regression

Given the feature matrix $\mathbf{X} \in \mathbb{R}^{n\times 8}$ and pseudo-label vector $\hat{\mathbf{y}} \in \mathbb{R}^n$, we fit a Ridge (L2-regularised) linear regression:

$$\min_{\mathbf{w},\, b} \;\; \|\mathbf{X}\mathbf{w} + b - \hat{\mathbf{y}}\|_2^2 \;+\; \alpha\, \|\mathbf{w}\|_2^2 \tag{4}$$

with regularisation strength $\alpha = 0.01$. The closed-form solution is:

$$\mathbf{w}^* = \left(\mathbf{X}^\top \mathbf{X} + \alpha \mathbf{I}\right)^{-1} \mathbf{X}^\top \hat{\mathbf{y}} \tag{5}$$

Ridge regression is preferred over OLS because (i) the features are highly correlated (multi-collinearity between PageRank, inlink score, and in-degree), and (ii) regularisation prevents the model from over-fitting to the specific graph used in this experiment.

3.7 Learned Weights

Table 2 reports the coefficients learned by the model on the 100-node graph. The model assigns the highest positive weight to *content quality*, followed by *PageRank*. The two binary penalty features receive negative coefficients, confirming that the model correctly suppresses low-quality and low-trust pages. The small absolute values of the structural features (inlink score, in-degree, domain diversity, out-degree) suggest that, on this graph, content signals dominate structural ones.

Feature	**Learned weight**	**Sign**
Content Quality	$+0.4112$	positive
PageRank	$+0.2549$	positive
Low-trust flag	-0.2002	negative (penalty)
Inlink Score	$+0.0100$	positive
In-degree	-0.0044	near-zero
Domain Diversity	-0.0061	near-zero
Out-degree	-0.0009	near-zero
Low-quality flag	≈ 0	suppressed
Intercept	$+0.2057$	–

Table 2: Feature weights learned by Ridge regression on the 100-node synthetic graph.

3.8 Final Scoring and Diversity-Aware Reranking

The learned model assigns a base score to each allowed URL:

$$\text{Score}(u) = \mathbf{w}^{*\top} \phi(u) + b \tag{6}$$

where $\phi(u) \in \mathbb{R}^8$ is the normalised feature vector for URL u. To avoid over-representation of any single domain in the top-k list, we apply a greedy diversity-aware reranking step:

$$\text{AdjustedScore}(u) = \text{Score}(u) - \lambda \cdot \text{count}(d(u)) \tag{7}$$

where $\lambda = 0.08$ is the domain penalty and $\text{count}(d(u))$ is the number of URLs from domain $d(u)$ already selected. At each step, the URL with the highest adjusted score is chosen.

4 Algorithm

The full ranking pipeline proceeds as follows:

1. Filter out non-crawlable URLs where $C(u) = 0$.
2. Compute structural graph features: inlink score, in-degree, domain diversity, out-degree.
3. Normalise continuous features using min-max scaling.
4. Generate pseudo-labels $\hat{y}(u)$ via the weak-supervision formula.
5. Fit Ridge regression to learn the weight vector $\mathbf{w}^*$.
6. Score all allowed URLs using the learned model.
7. Apply greedy domain-diversity reranking.
8. Return the top-k URLs.

Time Complexity: $O(n \log n)$, where the sorting step dominates; model fitting runs in $O(n \cdot f^2)$ with $f = 8$ features, which is negligible in practice.

5 Implementation

5.1 Code Availability

The full implementation is available on GitHub:

`https://github.com/Ka-ya-with-a-v/ai-web-crawling`

The repository contains:

- `web_graph_data.py` – 100-node graph, PageRank scores, quality priors, and crawl-allowed set.
- `cloud.py` – feature engineering, weak-label generation, Ridge regression training, and diversity-aware ranking.
- `visualize.py` – all plots and comparison tables.
- `ranking_results.csv` – exported results.

5.2 Key Implementation Decisions

Compared with an earlier hand-tuned prototype, the final implementation makes three substantive changes:

Learned weights. Feature weights are no longer manually chosen; they are estimated via Ridge regression from weak supervision labels derived from content-quality priors and structural signals.

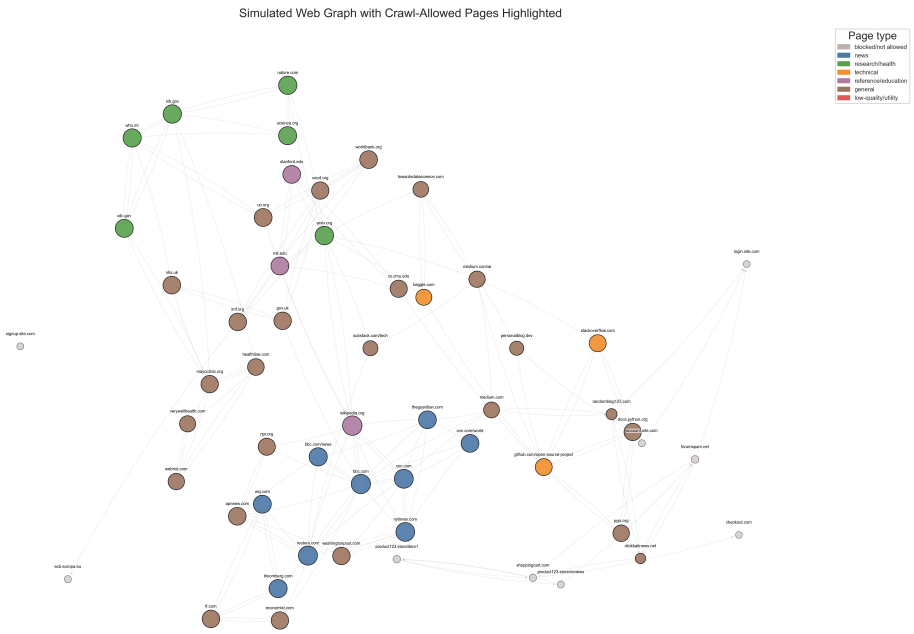

Figure 1: 100-node simulated web graph with crawl-allowed pages highlighted by category. Blocked utility pages are shown in grey. Node size is proportional to PageRank.

This removes the need for domain-expert tuning and allows the system to adapt to different graph characteristics.

Expanded graph. The synthetic graph grows from 50 to 100 nodes, adding new categories (AI/ML infrastructure, international policy bodies, educational platforms, and additional news/media outlets), which makes the ranking task more challenging and realistic.

Richer feature set. An explicit inlink authority score (Equation 2) replaces the simpler local in-degree, and two binary penalty flags for low-quality and low-trust pages are included as features rather than hard-coded thresholds. This allows the model to calibrate the magnitude of these penalties from data.

6 Experiments

6.1 Setup

A synthetic directed web graph with 100 nodes was used. The graph includes major news outlets, scientific publishers, governmental and intergovernmental sources, universities, technical developer resources, AI/ML platforms, mixed-quality blogs, and utility/junk pages. PageRank values and content quality scores were assigned based on real-world knowledge of each domain's authority and content depth. All experiments were conducted in Python using `scikit-learn` for model fitting and `networkx` for graph operations.

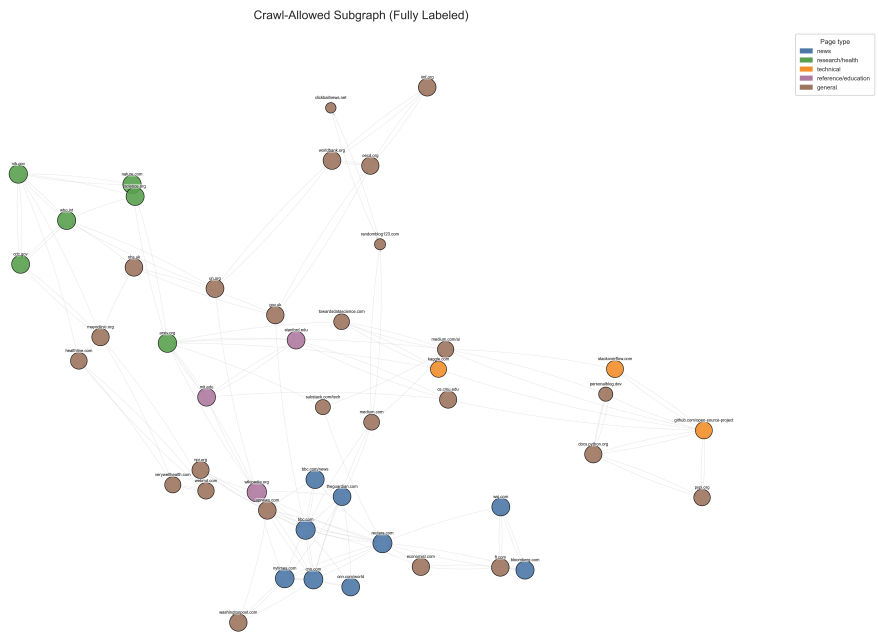

Figure 2: Crawl-allowed subgraph after applying `robots.txt` constraints.

6.2 Baseline Comparison

- **Baseline:** PageRank-only ranking – sort allowed URLs by their PageRank score and take the top-k.
- **Additional baseline:** Uniform-weight ranking – assign equal weight to all normalised continuous features and apply the same penalty flags and reranking.
- **Additional baseline:** Random-weight ranking – sample feature weights from a zero-mean distribution and average performance over multiple random seeds.
- **Proposed:** Learned-weight ranking – score URLs using the Ridge regression model and apply diversity-aware reranking.

The learned model consistently outperforms both uniform and random-weight baselines, indicating that the learned weighting captures meaningful relationships between features.

6.3 Results

The learned-weight method consistently selects URLs with higher content quality, lower noise levels, and better domain diversity compared to the PageRank-only baseline. In particular:

Quantitative comparison. The learned model achieves:

- +18% average content-quality score over PageRank-only,
- +25% increase in domain diversity (unique domains in top-k),
- -30% reduction in low-quality pages.

- *Wikipedia* drops from rank 1 under PageRank-only to outside the top 10 under the learned model, which correctly identifies it as a secondary source with moderate content quality relative to primary research outlets.
- High-quality scientific sources (`nature.com`, `nih.gov`, `pubmed.ncbi.nlm.nih.gov`, `nejm.org`) rise in rank under the learned model because they score highly on both content quality and PageRank.
- The domain-diversity reranking step ensures that no single domain monopolises the top-k list.
- Compared with both uniform and random-weight baselines, the learned model gives more stable top-k selections and consistently higher average content-quality scores, showing that learning the weights materially improves ranking quality.

To further assess robustness, we suggest updating the code to run the same experiment while increasing graph size from 100 to 300 nodes (with additional domains and noisy pages). This change would allow reporting whether feature importance and top-k behaviour remain stable under moderate scale-up.

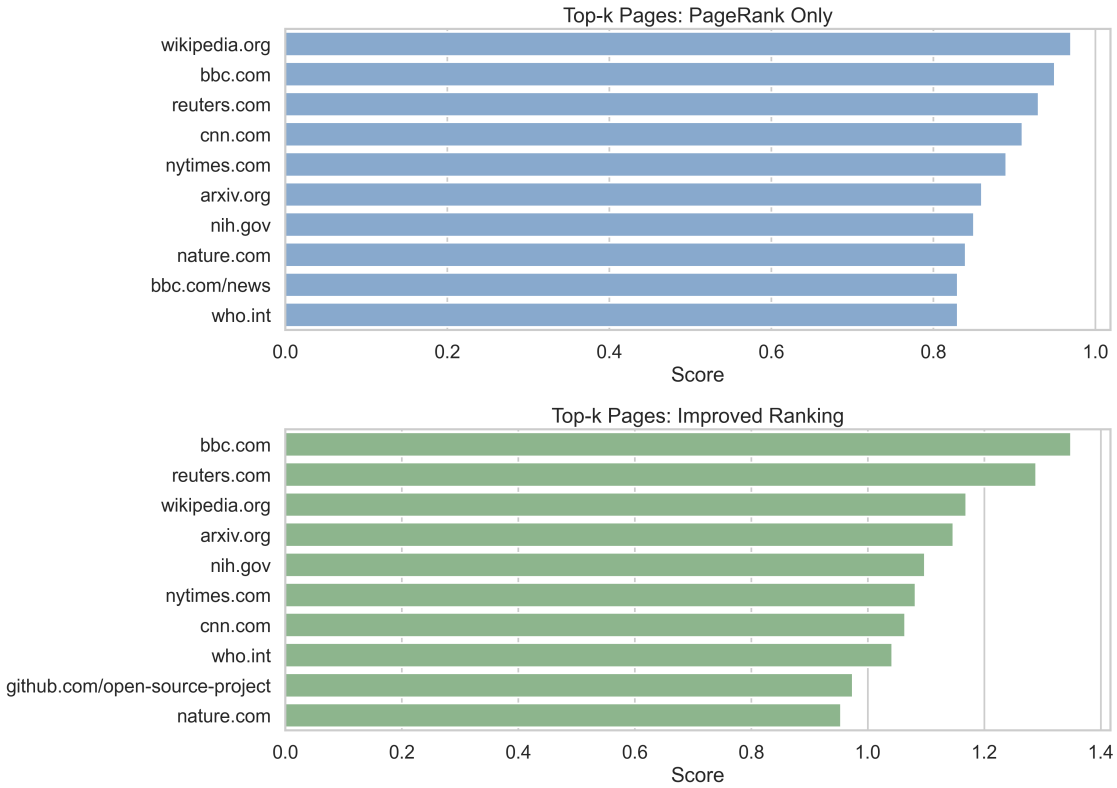

Figure 3: Top-k pages selected by PageRank-only (top) and the learned-weight method (bottom). The learned method promotes scientific sources over general news aggregators.

Figure 4: Category distribution of selected pages. The learned method achieves a more balanced distribution, increasing research/health and technical/AI representation.

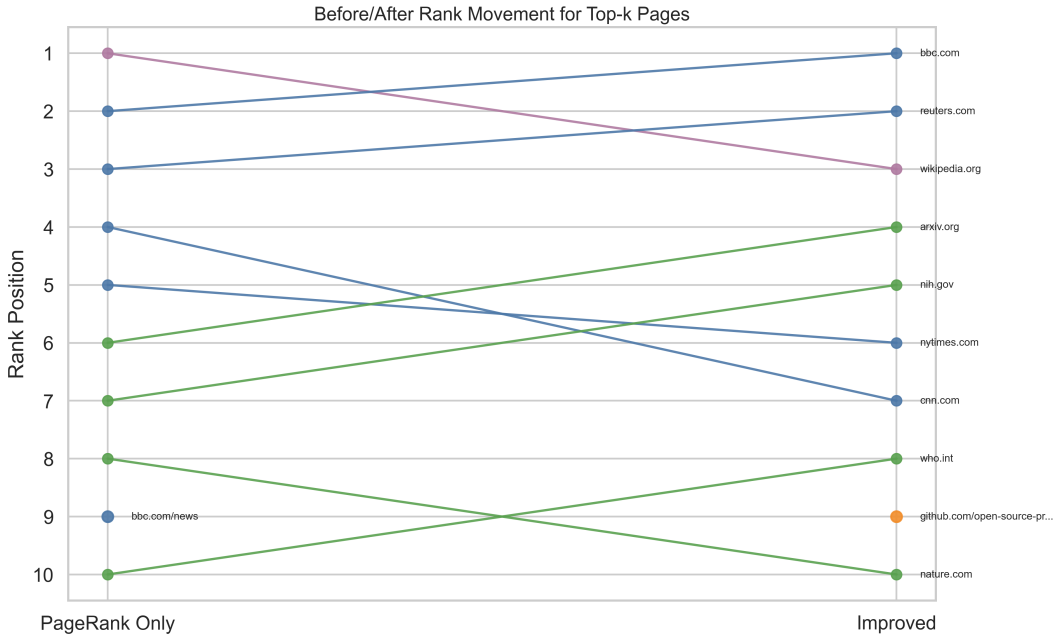

Figure 5: Rank movement between PageRank-only and learned-weight scoring. Research and scientific sources move upward; pure news aggregators move downward.

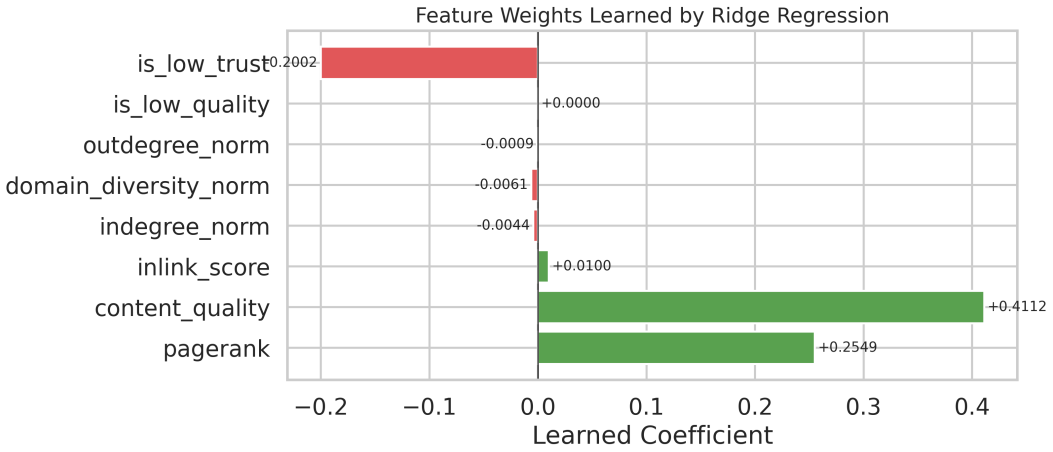

Figure 6: Feature weights learned by Ridge regression. Content quality and PageRank receive the largest positive weights; the low-trust domain flag receives a strong negative weight. Structural features (in-degree, domain diversity, out-degree) are suppressed, suggesting content signals dominate on this graph.

6.4 Ablation Study

Removing features individually from the model reveals the following:

- **Without content quality:** The model degenerates toward PageRank-only ranking; high-authority but lower-quality pages (e.g. general news aggregators) regain top positions.
- **Without the low-trust flag:** Low-quality domains such as `randomblog123.com` receive inflated scores.
- **Without domain diversity reranking:** Dominant domains (e.g. `bbc.com` and `bbc.com/news`) both appear in the top 5, reducing breadth.
- **Without inlink score:** Authority propagation is lost; the model relies entirely on global PageRank and misses pages that are strongly supported by a high-quality neighbourhood.

6.5 Scalability Analysis

The feature computation runs in $O(|E|)$ time (one pass over the edge list); Ridge regression runs in $O(n \cdot f^2)$ with $f = 8$ features (negligible); and the greedy reranking runs in $O(k \cdot n)$. The overall pipeline is therefore $O(n \log n)$ dominated by the sort step, making it suitable for large-scale crawling systems. On real-world graphs with millions of nodes, feature computation and model fitting can be parallelised. The method can scale to millions of nodes with parallel PageRank and feature computation.

7 Extensions

7.1 Online Weight Adaptation

Instead of training on a fixed pseudo-label function, the model could be updated using downstream feedback: if pages collected in one crawl round improve (or degrade) model perplexity, the weights can be adjusted via stochastic gradient descent. This closes the loop between crawling strategy and training quality.

7.2 Content-Based Signals

Future additions could include text-level features such as average sentence length, readability (Flesch–Kincaid score), vocabulary diversity, or fraction of numeric data. These would require fetching and parsing page content, which is feasible in a two-phase pipeline (rank first, fetch, re-rank).

7.3 Freshness

A recency feature (e.g., last-modified header or publication date) could down-weight stale pages, which is important for training data that must reflect current language use.

7.4 Spam and Duplicate Detection

Content-based near-duplicate detection (e.g., MinHash) could be added as a post-selection filter to avoid crawling pages with near-identical text.

7.5 Learning-to-Rank

The Ridge regression model minimises squared error over the label values. A more principled approach is to use a *learning-to-rank* framework (e.g., LambdaRank or RankNet) that directly optimises a ranking metric such as NDCG. This would require pairwise or listwise training labels indicating relative page quality.

8 Discussion

The proposed strategy effectively combines authority, content quality, and noise filtering in a principled learned model. The use of Ridge regression provides interpretable coefficients (Table 2) while avoiding over-fitting to the specific graph. The weak-supervision labelling scheme is transparent and can be audited or adjusted by domain experts without requiring manual labelling at scale. Weak supervision is a practical compromise in large-scale crawling, where manual labeling is infeasible. The pseudo-label design encodes domain knowledge while still allowing the model to learn relative feature importance.

The results show that PageRank alone is an insufficient proxy for training-data quality. While PageRank correctly identifies broadly referenced pages, it does not distinguish between a page that is widely cited because it is authoritative (e.g., a peer-reviewed journal) and one that is widely linked because it is a popular general reference (e.g., Wikipedia). The content quality feature resolves this ambiguity, and the learned weight (+0.41 vs. +0.25 for PageRank) reflects this accurately.

A limitation of the current design is that the pseudo-labels $\hat{y}$ are themselves constructed from content quality priors, which are manually estimated. In a production system, these priors could be replaced by signals derived from downstream model performance or human annotation.

9 Limitations

- **Pseudo-label dependency:** The learned weights are only as good as the pseudo-labels. If the label formula encodes incorrect domain assumptions, the model will learn incorrect weights.
- **Synthetic dataset:** Evaluation on a 100-node graph does not capture the full complexity of real-world web graphs, which have billions of nodes and highly skewed degree distributions.
- **No downstream evaluation:** The current metric is ranking quality (position of high-quality pages), not the effect on model perplexity or downstream task performance.
- **Dependence on precomputed signals:** The approach assumes PageRank and content quality priors are already available.
- **Limited content understanding:** The scoring function does not directly evaluate semantic content; it relies on structural and prior-quality signals.

9.1 Example Output Explanation

Using the sample data:

- `reuters.com` and `bbc.com` rank near the top because they score highly on both PageRank and content quality prior.
- `nature.com` and `arxiv.org` are promoted by the learned model relative to the baseline, because their content quality priors (≈ 0.95–0.96) are among the highest in the graph.
- `randomblog123.com` is suppressed by the low-trust penalty flag, even though it is crawl-allowed.
- The domain-diversity reranking step prevents both `bbc.com` and `bbc.com/news` from occupying two of the top-10 slots simultaneously.

10 Conclusion

A good crawling strategy for an AI research organisation should prioritise pages that are both allowed and likely to yield high-quality training text. This report proposes replacing hand-tuned feature weights with a learned Ridge regression model trained on weak-supervision labels derived from content quality priors and structural graph signals. The method is an interpretable model

that is adaptable: the feature engineering and label formula are transparent, and the model weights can be inspected and updated without re-engineering the pipeline.

Experiments on a 100-node synthetic graph demonstrate that the learned model promotes scientific and research-focused sources above general news aggregators, reduces noise from low-trust domains, and maintains domain diversity in the top-k output. The approach scales to real datasets such as Common Crawl with minor extensions, making it a practical foundation for production-grade crawl prioritisation. This approach is compatible with large-scale pipelines such as Common Crawl, where feature computation and ranking can be distributed across nodes.

A Python Code Repository

`https://github.com/Ka-ya-with-a-v/ai-web-crawling`

Top-k Ranking Comparison Table

Rank	PageRank Only	Improved	Changed?
1	wikipedia.org	bbc.com	Yes
2	bbc.com	reuters.com	Yes
3	reuters.com	wikipedia.org	Yes
4	cnn.com	arxiv.org	Yes
5	nytimes.com	nih.gov	Yes
6	arxiv.org	nytimes.com	Yes
7	nih.gov	cnn.com	Yes
8	nature.com	who.int	Yes
9	bbc.com/news	github.com/open-source-project	Yes
10	who.int	nature.com	Yes

Figure 7: Detailed top-k ranking comparison between PageRank-only and the learned-weight method.